

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)**

Department of Artificial Intelligence and Data Science

DSA0503 – Query Processing For Data Science

Lab Experiments 11-20

Name: Renee
Vincent ASR

**Register
Number:**192424161

11.Create a dataframe of ten rows, four columns with random values. Convert some values to nan values. Write a Pandas program which will highlight the nan values.

Highlight NaN Values in a DataFrame

Aim:

To highlight the NaN values in a DataFrame using Pandas.

Algorithm:

1. Import Pandas and NumPy.
2. Create a DataFrame with random values.
3. Convert some values to NaN.
4. Highlight and display the NaN values.

Code:

```
import pandas as pd
import numpy as np
import webbrowser
# Create DataFrame
np.random.seed(10)
df = pd.DataFrame(np.random.randn(10,4),
                  columns=['A','B','C','D'])
# Insert NaN values
df.iloc[0,2] = np.nan
df.iloc[3,1] = np.nan
df.iloc[5,3] = np.nan
df.iloc[8,0] = np.nan
# Highlight NaN values
def highlight_nan(x):
```

```

    return 'background-color:red' if pd.isna(x) else "
styled = df.style.map(highlight_nan)
# Save as HTML
styled.to_html("highlight_nan.html")
# Open automatically in browser
webbrowser.open("highlight_nan.html")
print("Output saved as highlight_nan.html")

```

Output:

	A	B	C	D
0	1.331587	0.715279	nan	-0.008384
1	0.621336	-0.720086	0.265512	0.108549
2	0.004291	-0.174600	0.433026	1.203037
3	-0.965066	nan	0.228630	0.445138
4	-1.136602	0.135137	1.484537	-1.079805
5	-1.977728	-1.743372	0.266070	nan
6	1.123691	1.672622	0.099149	1.397996
7	-0.271248	0.613204	-0.267317	-0.549309
8	nan	-0.476142	1.308473	0.195013
9	0.400210	-0.337632	1.256472	-0.731970

Result:

NaN values were highlighted successfully.

12.Create a dataframe of ten rows, four columns with random values. Write a Pandas program to set dataframe background Color black and font color yellow.

Set DataFrame Background Color Black and Font Color Yellow

Aim:

To apply black background and yellow font color to a DataFrame.

Algorithm:

1. Import Pandas and NumPy.
2. Create a DataFrame with random values.
3. Apply DataFrame styling.
4. Display the styled DataFrame.

Code:

```
import pandas as pd
import numpy as np
import webbrowser

# Create DataFrame with 10 rows and 4 columns
np.random.seed(10)
df = pd.DataFrame(
    np.random.randn(10, 4),
    columns=['A', 'B', 'C', 'D']
)

# Set background color black and font color yellow
styled_df = df.style.set_properties(**{
    'background-color': 'black',
    'color': 'yellow'
})

# Save as HTML
styled_df.to_html("black_yellow_dataframe.html")

# Open automatically in browser
webbrowser.open("black_yellow_dataframe.html")
print("Output saved as 'black_yellow_dataframe.html'")
```

Output:

	A	B	C	D
0	1.331587	0.715279	-1.545400	-0.008384
1	0.621336	-0.720086	0.265512	0.108549
2	0.004291	-0.174600	0.433026	1.203037
3	-0.965066	1.028274	0.228630	0.445138
4	-1.136602	0.135137	1.484537	-1.079805
5	-1.977728	-1.743372	0.266070	2.384967
6	1.123691	1.672622	0.099149	1.397996
7	-0.271248	0.613204	-0.267317	-0.549309
8	0.132708	-0.476142	1.308473	0.195013
9	0.400210	-0.337632	1.256472	-0.731970

Result:

The DataFrame was displayed with black background and yellow font color successfully.

13. Write a Pandas program to detect missing values of a given DataFrame. Display True or False.**Detect Missing Values in a DataFrame****Aim:**

To detect missing values in a DataFrame using Pandas.

Algorithm:

1. Import Pandas.
2. Create or read the DataFrame.
3. Use the isnull() function to detect missing values.
4. Display True or False for each value.

Code:

```
import pandas as pd
import numpy as np
# Create DataFrame
data = {
    'ord_no': [70001, np.nan, 70002, 70004, np.nan, 70005, np.nan, 70010, 70003, 70012,
              np.nan, 70013],
    'purch_amt': [150.50, 270.65, 65.26, 110.50, 948.50, 2400.60, 5760.00, 1983.43, 2480.40,
                  250.45, 75.29, 3045.60],
    'ord_date': ['2012-10-05', '2012-09-10', np.nan, '2012-08-17', '2012-09-10', '2012-07-27',
                 '2012-09-10', '2012-10-10', '2012-10-10', '2012-06-27', '2012-08-17', '2012-04-25'],
    'customer_id': [3002, 3001, 3001, 3003, 3002, 3001, 3001, 3004, 3003, 3002, 3001, 3001],
    'salesman_id': [5002, 5003, 5001, np.nan, 5002, 5001, 5001, np.nan, 5003, 5002, 5003,
                    np.nan]
}
df = pd.DataFrame(data)
print("Original DataFrame:\n")
print(df)
print("\nMissing Values (True = Missing, False = Not Missing):\n")
```

```
print(df.isnull())
```

Output:



The screenshot shows a Jupyter Notebook interface. The top part displays a DataFrame with columns: 'ord_no', 'purch_amt', 'ord_date', 'customer_id', and 'salesman_id'. The data rows show various orders with some missing values (NaN). Below the DataFrame, the output of the `df.isnull()` operation is shown, indicating which cells are missing (True) and which are not (False).

	ord_no	purch_amt	ord_date	customer_id	salesman_id
0	70001	0	2012-10-05	3002	5002
1	NaN	270.80	2012-09-10	3001	5003
2	70002	0	95-08	NaN	5001
3	70003	0	2012-08-17	3003	NaN
4	NaN	948.50	2012-05-10	3002	5002
5	70005	0	2012-07-27	3001	5001
6	NaN	3740.50	2012-09-10	3001	5001
7	70010	0	1993-43	2012-10-10	3004
8	70001	0	2490.40	2012-10-10	3003
9	70012	0	2302.45	2012-06-27	3002
10	NaN	75.20	2012-06-17	3001	5003
11	70013	0	3042.80	2012-04-25	3001

	ord_no	purch_amt	ord_date	customer_id	salesman_id
0	False	False	False	False	False
1	True	False	False	False	False
2	False	False	False	True	False
3	False	False	False	False	True
4	True	False	False	False	False
5	False	False	False	False	False
6	True	False	False	False	False
7	False	False	False	False	True
8	False	False	False	False	False
9	False	False	False	False	False
10	True	False	False	False	False
11	False	False	False	False	True

Result:

Missing values were detected and displayed successfully.

14. Write a Pandas program to find and replace the missing values in a given DataFrame which do not have any valuable information.

Find and Replace Missing Values in a DataFrame

Aim:

To find and replace invalid or missing values in a DataFrame.

Algorithm:

1. Import Pandas and NumPy.
2. Create the DataFrame.
3. Replace invalid values (?, --) with NaN.
4. Display the updated DataFrame.

Code:

```
import pandas as pd
import numpy as np

# Disable the FutureWarning behavior
pd.set_option('future.no_silent_downcasting', True)

# Create DataFrame

data = {
    'ord_no': [70001, np.nan, 70002, 70004, np.nan, 70005, '-', 70010, 70003, 70012, np.nan,
              70013],
```

```

'purch_amt': [150.5, 270.65, 65.26, 110.5, 948.5, 2400.6, 5760, '?', 12.43, 2480.4, 250.45,
3045.6],

'ord_date': ['?', '2012-09-10', np.nan, '2012-08-17', '2012-09-10',
'2012-07-27', '2012-09-10', '2012-10-10',
'2012-10-10', '2012-06-27', '2012-08-17', '2012-04-25'],

'customer_id': [3002, 3001, 3001, 3003, 3002, 3001, 3001, 3004, '--', 3002, 3001, 3001],

'salesman_id': [5002, 5003, '?', 5001, np.nan, 5002, 5001, '?', 5003, 5002, 5003, '--']
}

df = pd.DataFrame(data)

print("Original DataFrame:\n")

print(df)

# Replace missing indicators with NaN

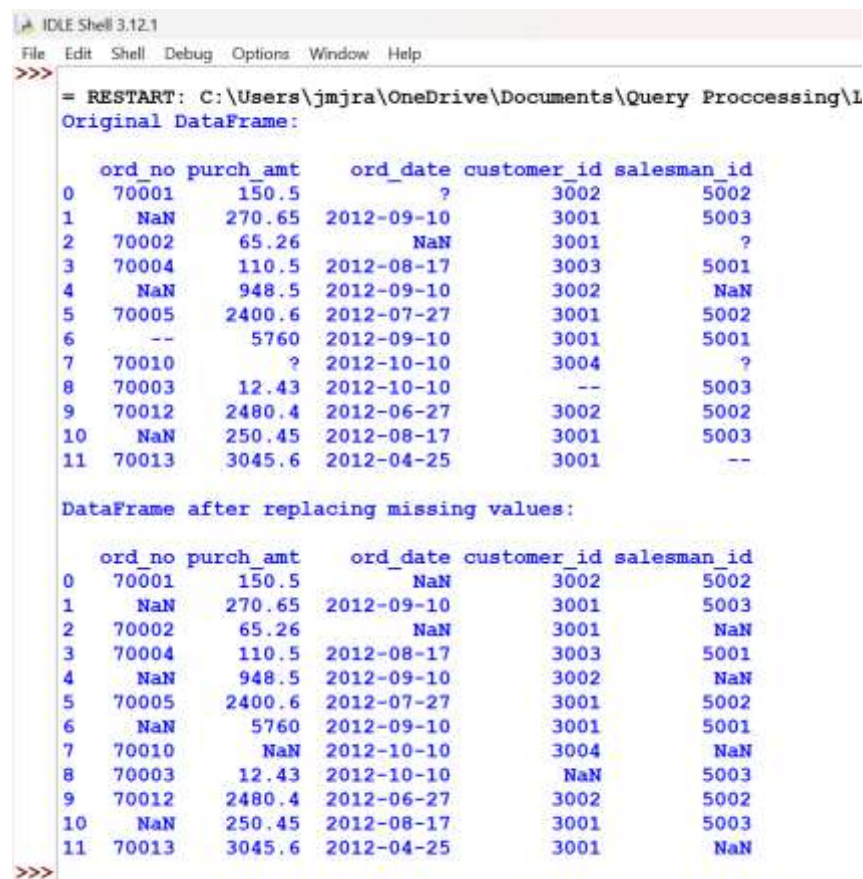
df = df.replace(['?', '--'], np.nan)

print("\nDataFrame after replacing missing values:\n")

print(df)

```

Output:



```

IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
>>> = RESTART: C:\Users\jmjra\OneDrive\Documents\Query Processing\I
Original DataFrame:
   ord_no purch_amt  ord_date customer_id salesman_id
0    70001    150.5         ?         3002         5002
1     NaN    270.65  2012-09-10         3001         5003
2    70002    65.26         NaN         3001          ?
3    70004    110.5  2012-08-17         3003         5001
4     NaN    948.5  2012-09-10         3002         NaN
5    70005   2400.6  2012-07-27         3001         5002
6     --    5760   2012-09-10         3001         5001
7    70010         ?  2012-10-10         3004          ?
8    70003    12.43  2012-10-10          --         5003
9    70012   2480.4  2012-06-27         3002         5002
10     NaN    250.45  2012-08-17         3001         5003
11   70013   3045.6  2012-04-25         3001          --

DataFrame after replacing missing values:
   ord_no purch_amt  ord_date customer_id salesman_id
0    70001    150.5         NaN         3002         5002
1     NaN    270.65  2012-09-10         3001         5003
2    70002    65.26         NaN         3001         NaN
3    70004    110.5  2012-08-17         3003         5001
4     NaN    948.5  2012-09-10         3002         NaN
5    70005   2400.6  2012-07-27         3001         5002
6     NaN    5760   2012-09-10         3001         5001
7    70010         NaN  2012-10-10         3004         NaN
8    70003    12.43  2012-10-10         NaN         5003
9    70012   2480.4  2012-06-27         3002         5002
10     NaN    250.45  2012-08-17         3001         5003
11   70013   3045.6  2012-04-25         3001         NaN
>>>

```

Result:

Missing and invalid values were replaced successfully.

15. Write a Pandas program to keep the rows with at least 2 NaN values in a given DataFrame.**Keep Rows with at Least Two NaN Values****Aim:**

To display only the rows containing at least two NaN values.

Algorithm:

1. Import Pandas and NumPy.
2. Create the DataFrame.
3. Count the NaN values in each row.
4. Display rows having two or more NaN values.

Code:

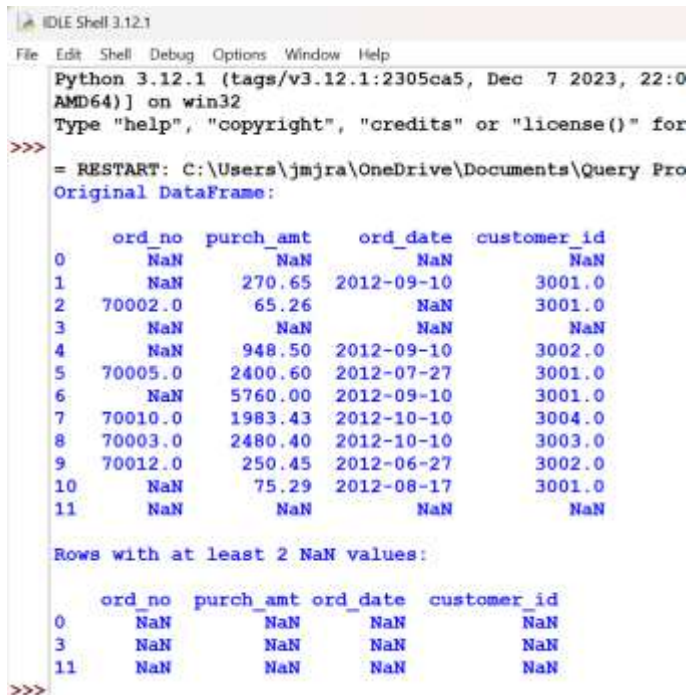
```
import pandas as pd
import numpy as np
# Create DataFrame
data = {
    'ord_no': [np.nan, np.nan, 70002, np.nan, np.nan, 70005, np.nan, 70010, 70003, 70012,
              np.nan, np.nan],
    'purch_amt': [np.nan, 270.65, 65.26, np.nan, 948.50, 2400.60, 5760.00, 1983.43, 2480.40,
                  250.45, 75.29, np.nan],
    'ord_date': [np.nan, '2012-09-10', np.nan, np.nan, '2012-09-10', '2012-07-27',
                 '2012-09-10', '2012-10-10', '2012-10-10', '2012-06-27',
                 '2012-08-17', np.nan],
    'customer_id': [np.nan, 3001, 3001, np.nan, 3002, 3001,
                    3001, 3004, 3003, 3002, 3001, np.nan]
}
# Create DataFrame
df = pd.DataFrame(data)
print("Original DataFrame:\n")
print(df)
```

```
# Keep rows with at least 2 NaN values
result = df[df.isnull().sum(axis=1) >= 2]

print("\nRows with at least 2 NaN values:\n")

print(result)
```

Output:



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:0
AMD64) on win32
Type "help", "copyright", "credits" or "license()" for
>>>
= RESTART: C:\Users\jmjra\OneDrive\Documents\Query Pro
Original DataFrame:
   ord_no  purch_amt  ord_date  customer_id
0      NaN      NaN      NaN      NaN
1      NaN      270.65  2012-09-10      3001.0
2  70002.0      65.26      NaN      3001.0
3      NaN      NaN      NaN      NaN
4      NaN      948.50  2012-09-10      3002.0
5  70005.0     2400.60  2012-07-27      3001.0
6      NaN     5760.00  2012-09-10      3001.0
7  70010.0     1983.43  2012-10-10      3004.0
8  70003.0     2480.40  2012-10-10      3003.0
9  70012.0      250.45  2012-06-27      3002.0
10     NaN       75.29  2012-08-17      3001.0
11     NaN      NaN      NaN      NaN

Rows with at least 2 NaN values:
   ord_no  purch_amt  ord_date  customer_id
0      NaN      NaN      NaN      NaN
3      NaN      NaN      NaN      NaN
11     NaN      NaN      NaN      NaN
>>>
```

Result:

Rows containing at least two NaN values were displayed successfully.

16. Write a Pandas program to split the following dataframe into groups based on school code. Also check the type of GroupBy object.

Split DataFrame into Groups Based on School Code

Aim:

To split the DataFrame into groups based on school code using Pandas.

Algorithm:

1. Import the Pandas library.
2. Create the DataFrame.
3. Group the data by the school column.
4. Display the groups and the GroupBy object type.

Code:

```
import pandas as pd
```



```

# Create DataFrame
data = {
    'school': ['s001', 's002', 's003', 's001', 's002', 's004'],
    'class': ['V', 'V', 'VI', 'VI', 'V', 'VI'],
    'name': ['Alberto Franco', 'Gino Mcneill', 'Ryan Parkes',
            'Eesha Hinton', 'Gino Mcneill', 'David Parkes'],
    'date_of_Birth': ['15/05/2002', '17/05/2002', '16/02/1999',
                     '25/09/1998', '11/05/2002', '15/09/1997'],
    'age': [12, 12, 13, 13, 14, 12],
    'height': [173, 192, 186, 167, 151, 159],
    'weight': [35, 32, 33, 30, 31, 32],
    'address': ['street1', 'street2', 'street3',
               'street1', 'street2', 'street4']
}

# Create DataFrame with index labels
df = pd.DataFrame(data, index=['S1', 'S2', 'S3', 'S4', 'S5', 'S6'])
print("Original DataFrame:\n")
print(df)

# Group by school
grouped = df.groupby('school')

# Display type of GroupBy object
print("\nType of GroupBy Object:")
print(type(grouped))

# Display each group
print("\nGroups based on School Code:\n")
for school, group in grouped:
    print("School:", school)
    print(group)
    print()

```

Output:

```
Python 3.12.1
File Edit Shell Debug Options Window Help
> RESTART: C:\Users\jmrja\OneDrive\Documents\Query Processing\Lab-15.py
Original DataFrame:
  school class      name date of Birth  age  height  weight  address
S1  s001     V  Alberto Franco   15/05/2002  12    173    35  street1
S2  s002     V    Gino McNeill   17/05/2002  12    192    32  street2
S3  s003     VI   Ryan Parkes   16/02/1999  13    186    33  street3
S4  s001     VI   Eesha Hinton   25/09/1998  13    167    30  street1
S5  s002     V    Gino McNeill   11/05/2002  14    151    31  street2
S6  s004     VI   David Parkes   15/09/1997  12    159    32  street4

Type of GroupBy Object:
<class 'pandas.core.groupby.generic.DataFrameGroupBy'>

Groups based on School Code:

School: s001
  school class      name date of Birth  age  height  weight  address
S1  s001     V  Alberto Franco   15/05/2002  12    173    35  street1
S4  s001     VI   Eesha Hinton   25/09/1998  13    167    30  street1

School: s002
  school class      name date of Birth  age  height  weight  address
S2  s002     V    Gino McNeill   17/05/2002  12    192    32  street2
S5  s002     V    Gino McNeill   11/05/2002  14    151    31  street2

School: s003
  school class      name date of Birth  age  height  weight  address
S3  s003     VI   Ryan Parkes   16/02/1999  13    186    33  street3

School: s004
  school class      name date of Birth  age  height  weight  address
S6  s004     VI   David Parkes   15/09/1997  12    159    32  street4
```

Result:

The DataFrame was successfully grouped based on school code.

17. Write a Pandas program to split the following dataframe by school code and get mean, min, and max value of age for each school.

Find Mean, Minimum, and Maximum Age for Each School

Aim:

To calculate the mean, minimum, and maximum age for each school.

Algorithm:

1. Import the Pandas library.
2. Create the DataFrame.
3. Group the data by the school column.
4. Calculate and display the mean, minimum, and maximum age.

Code:

```
import pandas as pd

# Create DataFrame
data = {
    'school': ['s001', 's002', 's003', 's001', 's002', 's004'],
    'class': ['V', 'V', 'VI', 'VI', 'V', 'VI'],
```

```

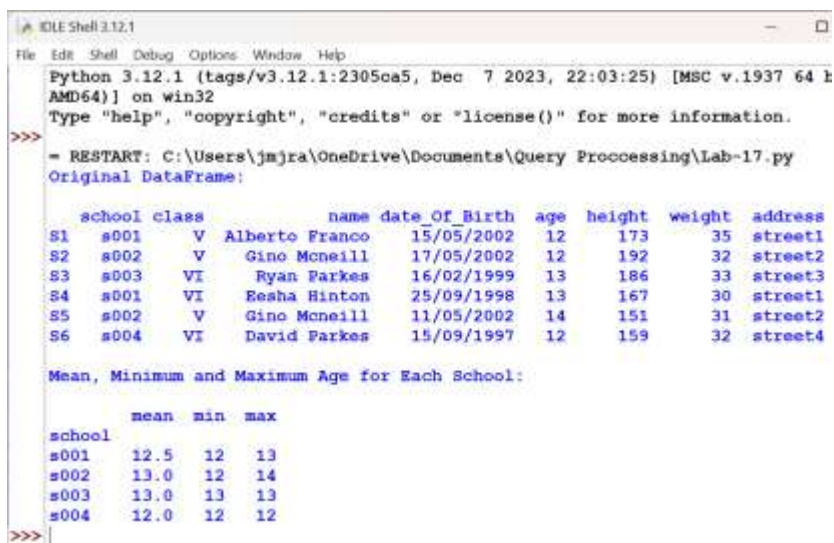
'name': ['Alberto Franco', 'Gino Mcneill', 'Ryan Parkes',
        'Eesha Hinton', 'Gino Mcneill', 'David Parkes'],
'date_Of_Birth': ['15/05/2002', '17/05/2002', '16/02/1999',
                  '25/09/1998', '11/05/2002', '15/09/1997'],
'age': [12, 12, 13, 13, 14, 12],
'height': [173, 192, 186, 167, 151, 159],
'weight': [35, 32, 33, 30, 31, 32],
'address': ['street1', 'street2', 'street3',
            'street1', 'street2', 'street4']
}

# Create DataFrame
df = pd.DataFrame(data, index=['S1', 'S2', 'S3', 'S4', 'S5', 'S6'])
print("Original DataFrame:\n")
print(df)

# Group by school and calculate mean, min and max age
result = df.groupby('school')['age'].agg(['mean', 'min', 'max'])
print("\nMean, Minimum and Maximum Age for Each School:\n")
print(result)

```

Output:



```

Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 b
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
- RESTART: C:\Users\jnjra\OneDrive\Documents\Query Proccessing\Lab-17.py
Original DataFrame:
  school class  name date_Of_Birth  age  height  weight  address
S1  s001     V  Alberto Franco  15/05/2002   12    173     35  street1
S2  s002     V   Gino Mcneill  17/05/2002   12    192     32  street2
S3  s003     VI   Ryan Parkes  16/02/1999   13    186     33  street3
S4  s001     VI   Eesha Hinton  25/09/1998   13    167     30  street1
S5  s002     V   Gino Mcneill  11/05/2002   14    151     31  street2
S6  s004     VI   David Parkes  15/09/1997   12    159     32  street4

Mean, Minimum and Maximum Age for Each School:
  school  mean  min  max
s001    12.5   12   13
s002    13.0   12   14
s003    13.0   13   13
s004    12.0   12   12
>>>

```

Result:

The mean, minimum, and maximum age for each school were displayed successfully.

18. Write a Pandas program to split the following given dataframe into groups based on school code and class.

Split DataFrame into Groups Based on School Code and Class

Aim:

To split the DataFrame into groups based on school code and class.

Algorithm:

1. Import the Pandas library.
2. Create the DataFrame.
3. Group the data by school and class.
4. Display the grouped data.

Code:

```
import pandas as pd

# Create DataFrame
data = {
    'school': ['s001', 's002', 's003', 's001', 's002', 's004'],
    'class': ['V', 'V', 'VI', 'VI', 'V', 'VI'],
    'name': ['Alberto Franco', 'Gino Mcneill', 'Ryan Parkes',
            'Eesha Hinton', 'Gino Mcneill', 'David Parkes'],
    'date_of_birth': ['15/05/2002', '17/05/2002', '16/02/1999',
                    '25/09/1998', '11/05/2002', '15/09/1997'],
    'age': [12, 12, 13, 13, 14, 12],
    'height': [173, 192, 186, 167, 151, 159],
    'weight': [35, 32, 33, 30, 31, 32],
    'address': ['street1', 'street2', 'street3',
               'street1', 'street2', 'street4']
}

# Create DataFrame
df = pd.DataFrame(data, index=['S1', 'S2', 'S3', 'S4', 'S5', 'S6'])
print("Original DataFrame:\n")
print(df)

# Group by school and class
```

```

grouped = df.groupby(['school', 'class'])

print("\nGroups based on School Code and Class:\n")

# Display each group

for (school, cls), group in grouped:

    print("School:", school, " Class:", cls)

    print(group)

    print()

```

Output:

The screenshot shows a Jupyter Notebook interface. The top part displays the 'Original DataFrame' with 8 rows and 8 columns: school, class, name, date_of_birth, age, height, weight, and address. The data is as follows:

	school	class	name	date_of_birth	age	height	weight	address
S1	s001	V	Alberto Franco	15/05/2002	12	173	35	street1
S2	s002	V	Gino Mcneill	17/05/2002	12	192	32	street2
S3	s003	VI	Ryan Parkes	16/02/1999	13	186	33	street3
S4	s001	VI	Easha Hinton	25/09/1998	13	167	30	street1
S5	s002	V	Gino Mcneill	11/05/2002	14	151	31	street2
S6	s004	VI	David Parkes	15/09/1997	12	159	32	street4

The bottom part of the screenshot shows the 'Groups based on School Code and Class' output. It displays the grouped data for each school and class combination:

- School: s001 Class: V**

school	class	name	date_of_birth	age	height	weight	address	
S1	s001	V	Alberto Franco	15/05/2002	12	173	35	street1
- School: s001 Class: VI**

school	class	name	date_of_birth	age	height	weight	address	
S4	s001	VI	Easha Hinton	25/09/1998	13	167	30	street1
- School: s002 Class: V**

school	class	name	date_of_birth	age	height	weight	address	
S2	s002	V	Gino Mcneill	17/05/2002	12	192	32	street2
S5	s002	V	Gino Mcneill	11/05/2002	14	151	31	street2
- School: s003 Class: VI**

school	class	name	date_of_birth	age	height	weight	address	
S3	s003	VI	Ryan Parkes	16/02/1999	13	186	33	street3
- School: s004 Class: VI**

school	class	name	date_of_birth	age	height	weight	address	
S6	s004	VI	David Parkes	15/09/1997	12	159	32	street4

Result:

The DataFrame was successfully grouped based on school code and class.

19. Write a Pandas program to display the dimensions or shape of the World alcohol consumption dataset. Also extract the column names from the dataset.

Display the Shape and Column Names of the World Alcohol Dataset

Aim:

To display the dimensions and column names of the World Alcohol Consumption dataset.

Algorithm:

1. Import the Pandas library.
2. Read the world_alcohol.csv file.
3. Display the shape of the DataFrame.
4. Display the column names.

Input CSV File:world_alcohol.csv

Year	WHO region	Country	Beverage Types	Display Value
1986	Western Pacific	Viet Nam	Wine	0
1986	Americas	Uruguay	Other	0.5
1985	Africa	Cte d'Ivoire	Wine	1.62
1986	Americas	Colombia	Beer	4.27
1987	Americas	Saint Kitts and Nevis	Beer	1.98
1987	Europe	France	Wine	12.3
1986	Europe	Germany	Beer	9.8
1985	South-East Asia	India	Spirits	1.2
1986	Eastern Mediterranean	Egypt	Other	0.1
1987	Western Pacific	Japan	Sake	7.5
1985	Europe	Italy	Wine	10.5
1986	Americas	Canada	Beer	6.3
1987	Africa	Nigeria	Beer	3.4
1985	Europe	Spain	Wine	11.2
1986	South-East Asia	Sri Lanka	Spirits	2.1

Code:

```
import pandas as pd

# Read the dataset
df = pd.read_csv("world_alcohol.csv")

# Display the dataset
print("World Alcohol Consumption Dataset:\n")
print(df)

# Display the dimensions (shape) of the dataset
print("\nDimensions (Rows, Columns):")
print(df.shape)

# Display the column names
print("\nColumn Names:")
print(df.columns)
```

Output:

```
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:\Users\jnjra\OneDrive\Documents\Query Processing\Lab-19.py
World Alcohol Consumption Dataset:

   Year  WHO region  ... Beverage Types Display Value
0  1986  Western Pacific  ...      Wine      0.00
1  1986  Americas  ...      Other      0.50
2  1985  Africa  ...      Wine      1.62
3  1986  Americas  ...      Beer      4.27
4  1987  Americas  ...      Beer      1.98
5  1987  Europe  ...      Wine     12.30
6  1986  Europe  ...      Beer      9.80
7  1985  South-East Asia  ...  Spirits      1.20
8  1986  Eastern Mediterranean  ...  Other      0.10
9  1987  Western Pacific  ...      Sake      7.50
10 1985  Europe  ...      Wine     10.50
11 1986  Americas  ...      Beer      6.30
12 1987  Africa  ...      Beer      3.40
13 1985  Europe  ...      Wine     11.20
14 1986  South-East Asia  ...  Spirits      2.10

[15 rows x 5 columns]

Dimensions (Rows, Columns):
(15, 5)

Column Names:
Index(['Year', 'WHO region', 'Country', 'Beverage Types', 'Display Value'], dtype
e='object')
>>>
```

Result:

The dataset dimensions and column names were displayed successfully.

20. Write a Pandas program to find the index of a given substring of a DataFrame column.

Find the Index of a Given Substring in a DataFrame Column

Aim:

To find the index of a specified substring in a DataFrame column.

Algorithm:

1. Import the Pandas library.
2. Read the world_alcohol.csv file.
3. Search for the specified substring in a column.
4. Display the matching row index.

Input CSV File: world_alcohol.csv

Code:

```
import pandas as pd
# Read the CSV file
df = pd.read_csv("world_alcohol.csv")
print("World Alcohol Consumption Dataset:\n")
```

```

print(df)

# Find the index of rows where Country contains 'Nam'

index = df[df['Country'].str.contains('Nam', case=False, na=False)].index

print("\nIndex of rows containing the substring 'Nam' in the Country column:")

print(index.tolist())

```

Output:

```

Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more informat
>>>
= RESTART: C:\Users\jmrja\OneDrive\Documents\Query Processing\Lab-2\
World Alcohol Consumption Dataset:

   Year  WHO region  ... Beverage Types Display Value
0  1986  Western Pacific  ...      Wine           0.00
1  1986      Americas  ...      Other           0.50
2  1985      Africa  ...      Wine           1.62
3  1986      Americas  ...      Beer           4.27
4  1987      Americas  ...      Beer           1.98
5  1987      Europe  ...      Wine          12.30
6  1986      Europe  ...      Beer           9.80
7  1985  South-East Asia  ...  Spirits           1.20
8  1986  Eastern Mediterranean  ...      Other           0.10
9  1987  Western Pacific  ...      Sake           7.50
10 1985      Europe  ...      Wine          10.50
11 1986      Americas  ...      Beer           6.30
12 1987      Africa  ...      Beer           3.40
13 1985      Europe  ...      Wine          11.20
14 1986  South-East Asia  ...  Spirits           2.10

[15 rows x 5 columns]

Index of rows containing the substring 'Nam' in the Country column:
[0]
>>>

```

Result:

The index of the specified substring was displayed successfully.